CH5 機器學習的基礎

主講：邱偉豪

1 普適化與優化的權衡

2 模型的評估方法

3 提升模型擬合（Fitting）

4 達到更佳的普適性

相關檔案

- Google Colab
- Deep Learning with Python

機器學習的終極目標：普適化

核心衝突

- 最佳化（Optimization）：調整模型以在「訓練資料」上表現最好
- 普適化（Generalization）：模型在「前所未見的資料」上的表現

深度學習的本質

- 深度學習透過學習訓練樣本的潛在流形（Latent Manifold）來運作。
- 具備普適化能力，代表模型學會了在樣本之間進行內插（Interpolate），而非單純記憶。
- 限制： 只能處理與訓練樣本接近的輸入，無法進行極端的外推。

何謂流形（Manifold）

數據在原始狀態下的樣子

通過層層轉換的數據

內插（Interpolation）

流形內插法
(潛在流形空間上的中間點)

線性內插法
(直接在編碼空間上進行算術平均)

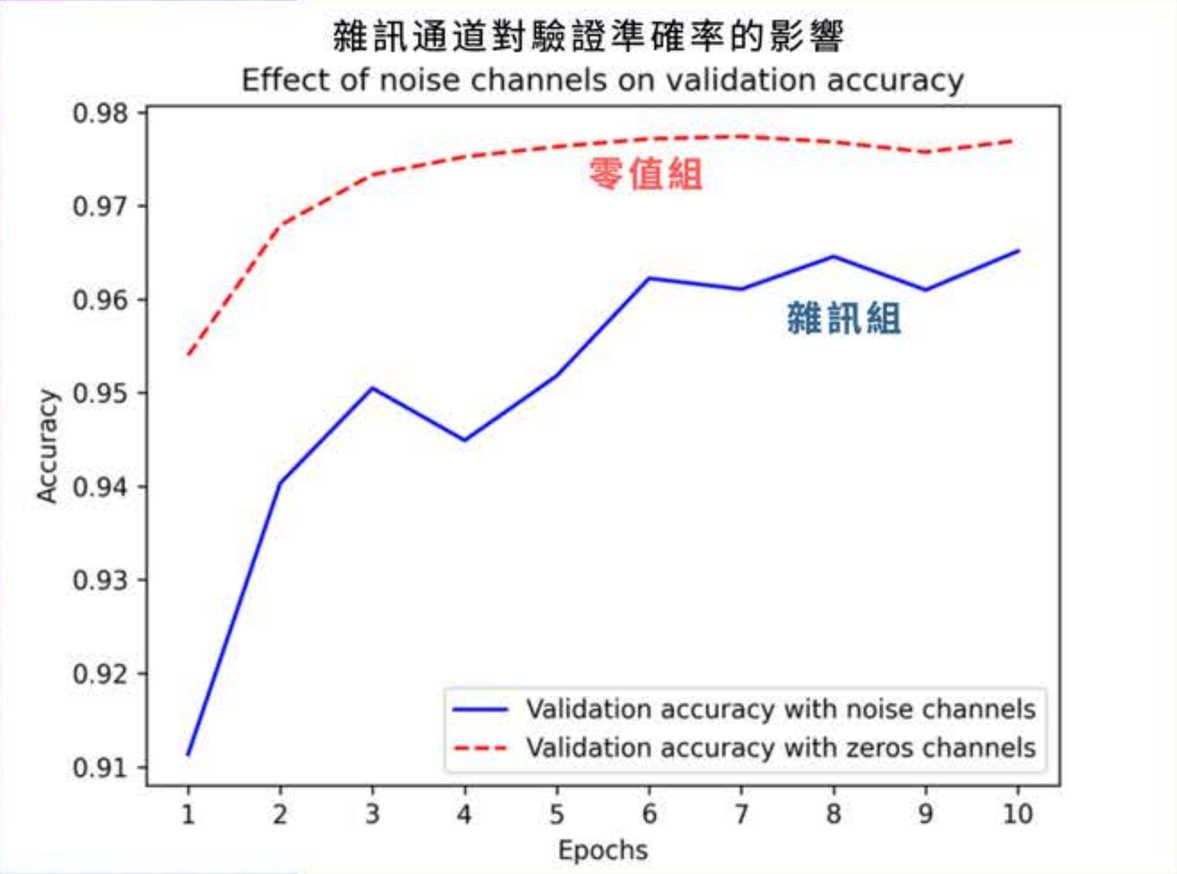
實驗一：雜訊對模型的影響 (MNIST)

實驗設計

- 在 MNIST 數據中加入「白雜訊」與「全零通道」

觀察結果

- 帶有雜訊的數據更容易導致驗證準確度下降。
- 模型會嘗試去「記憶」這些無意義的雜訊（將雜訊納入流形的一部分），導致普適化能力變差。



該實驗的結果圖表

- 藍線（加入雜訊）的驗證準確率明顯低於紅線（加入全零通道）
- 這張圖直接證明了雜訊會破壞模型的學習，導致驗證效能下降

實驗二：深度學習模型的強大記憶力

極端測試

- 將 MNIST 的標籤全部隨機打亂，訓練模型

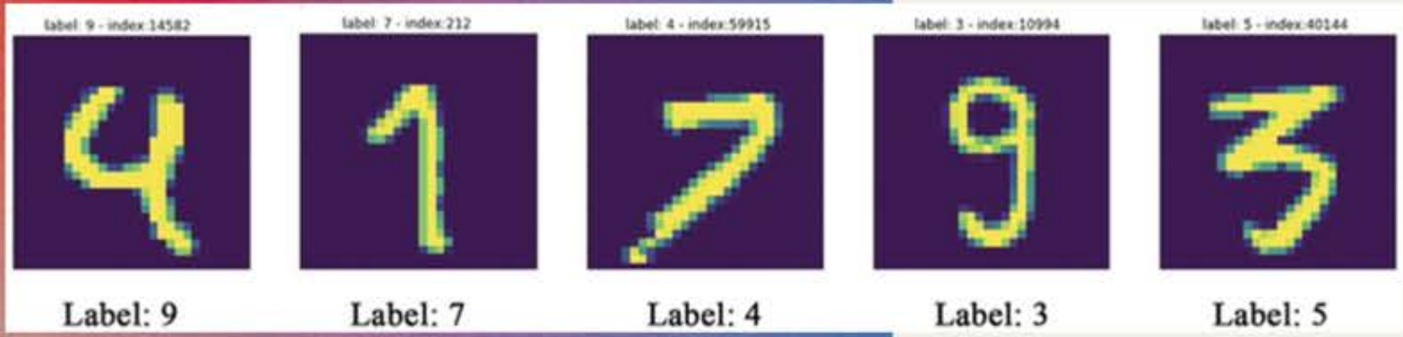
觀察結果

- 訓練準確度： 依然上升（模型硬背下了隨機標籤）。
- 驗證準確度： 停留在隨機猜測水平。

結論

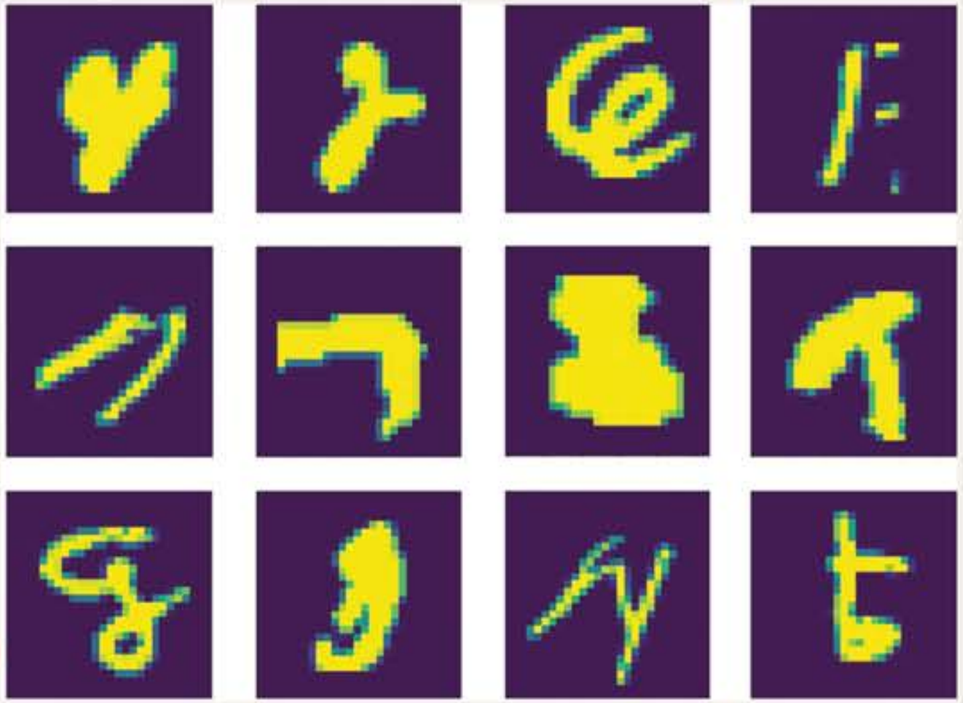
- 只要模型夠大，它能記住任何數據。因此目標不只是降低 Training Loss，而是確保學到的是規律。

標籤錯誤的數字



雖然這些不是隨機打亂標籤實驗的直接數據圖，但這兩張圖展示了髒數據的樣貌。
當數據標籤錯誤或數據本身極端異常時，模型如果強行去記憶這些錯誤，就會失去普適化能力。

怪異的數字



評估機器學習模型

為什麼需要評估?

- 為了監控普適化能力，避免資訊洩漏 (Information Leak)

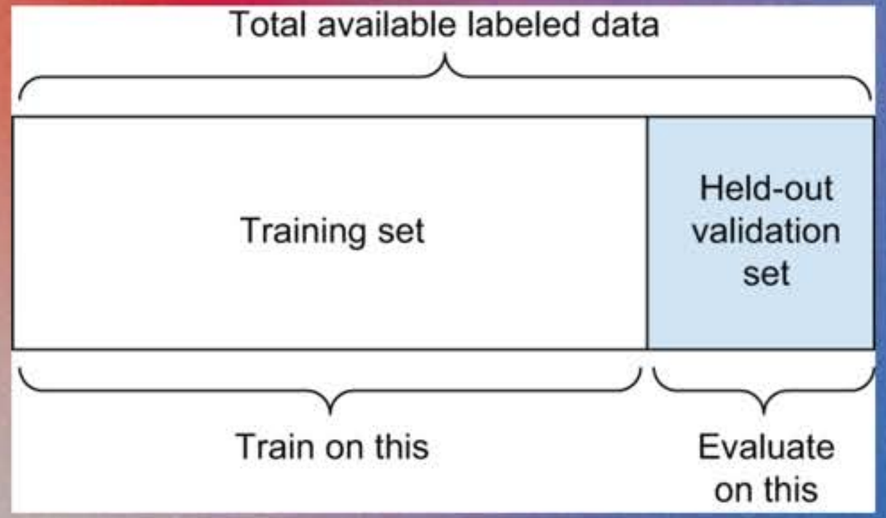
評估策略 (依數據量選擇)

- 簡單拆分(Holdout Validation)：適用於數據量充足時
- K 折交叉驗證(K-fold Cross-Validation)：適用於數據量較少，能更穩定評估模型表現
- 多次洗牌 K 折(Iterated K-fold with shuffling)：追求極致精確的評估

黃金法則

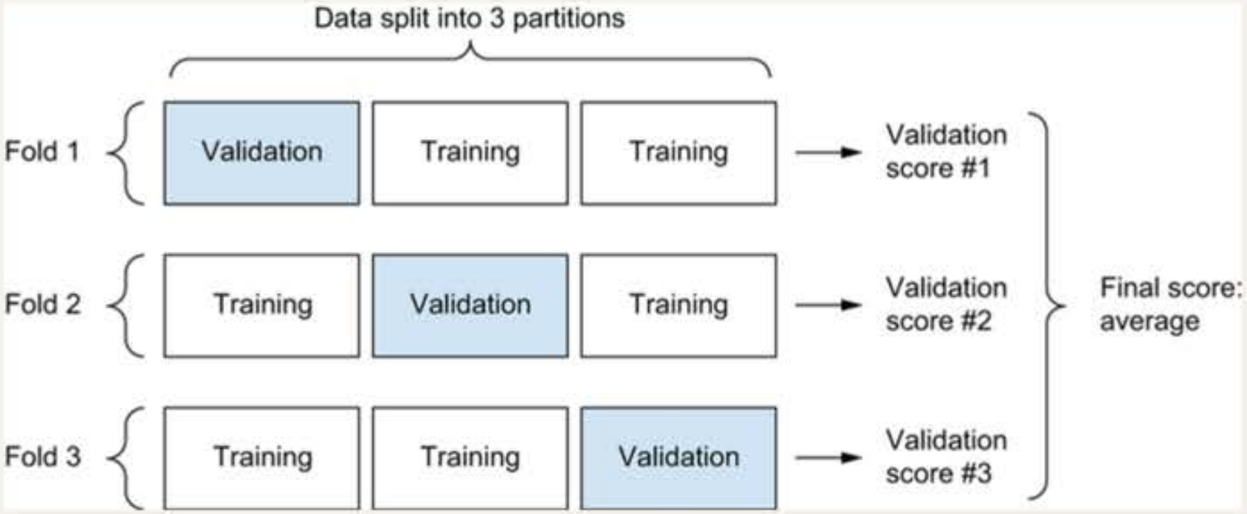
- 測試集 (Test Set) 只能在最後使用一次

簡單拆分(Holdout Validation)



數據量充足的情境下，切出 20% 左右作為驗證集

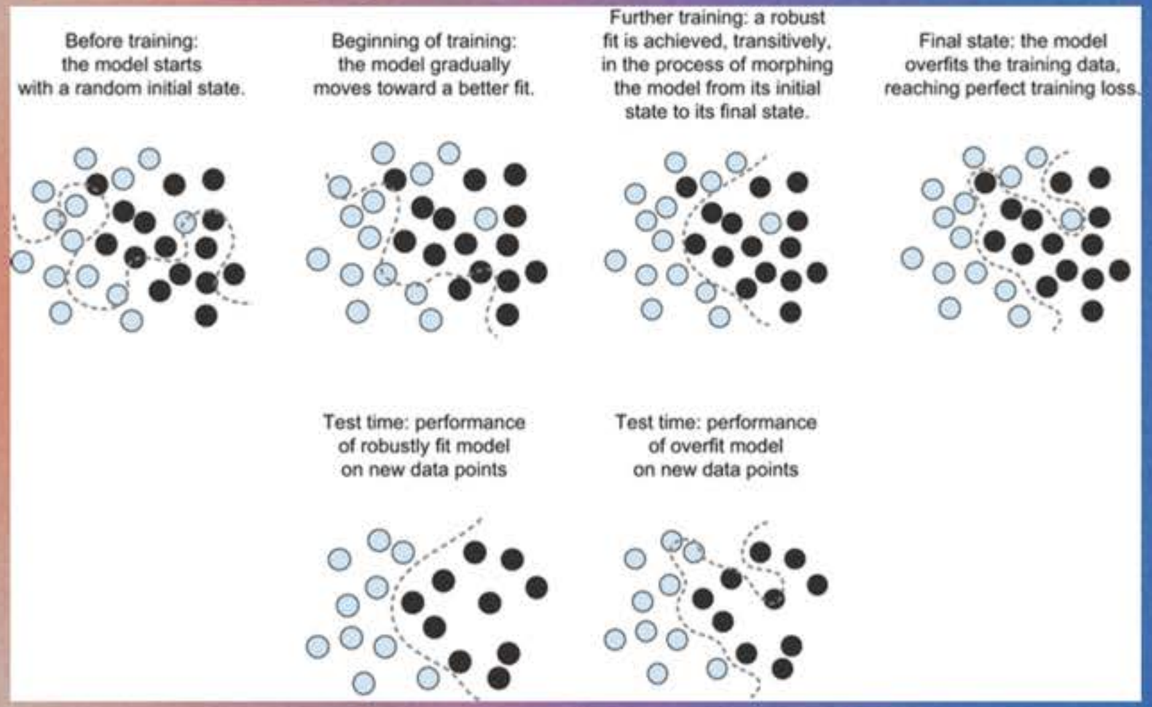
K 折交叉驗證(K-fold Cross-Validation)



數據量較少的情境下，輪流當驗證集，取平均分數，結果會更穩定可靠

提升模型的擬合表現
(克服 Underfitting)

擬合過程示意圖



- 從左邊的隨機亂猜到中間的逐漸成形，這就是調整學習率與批次量想要達到的效果，也就是推動決策邊界去擬合數據點
- 如果參數設錯，這條線就不會動

問題

- 訓練開始時，若連 Training Loss 都降不下來?

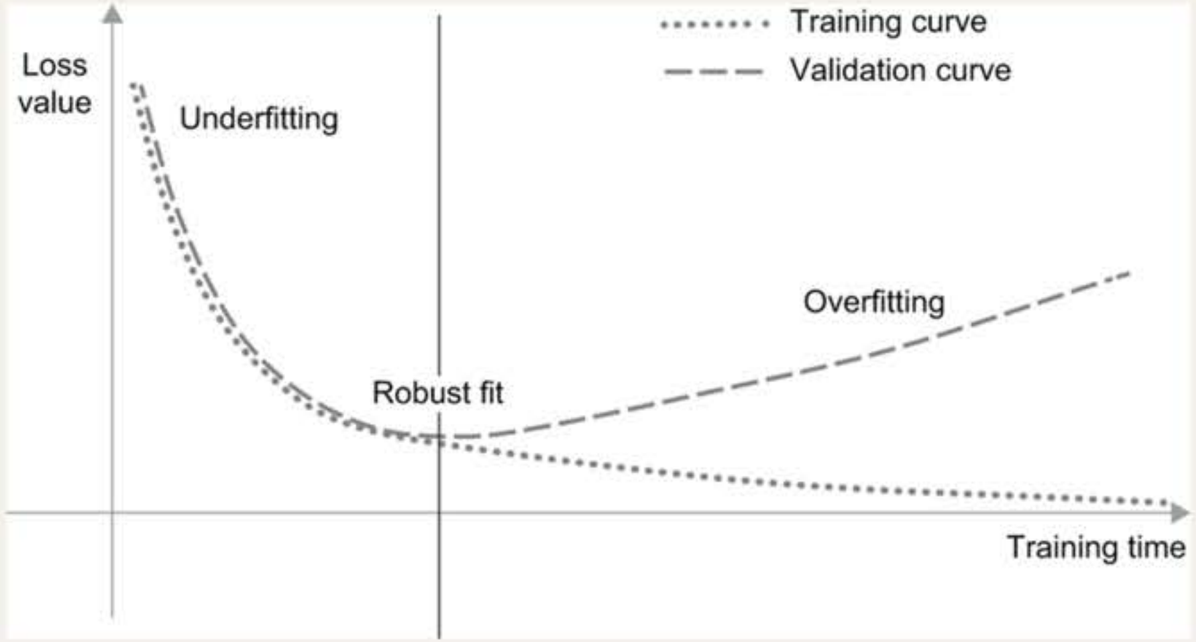
調整方向 1： 梯度下降參數

- 學習率 (Learning Rate)： 過大會震盪，過小則收斂太慢
- 批次量 (Batch Size)： 影響梯度更新的穩定性

調整方向 2： 增加訓練時間

- 給予模型更多的迭代次數來尋找最佳解

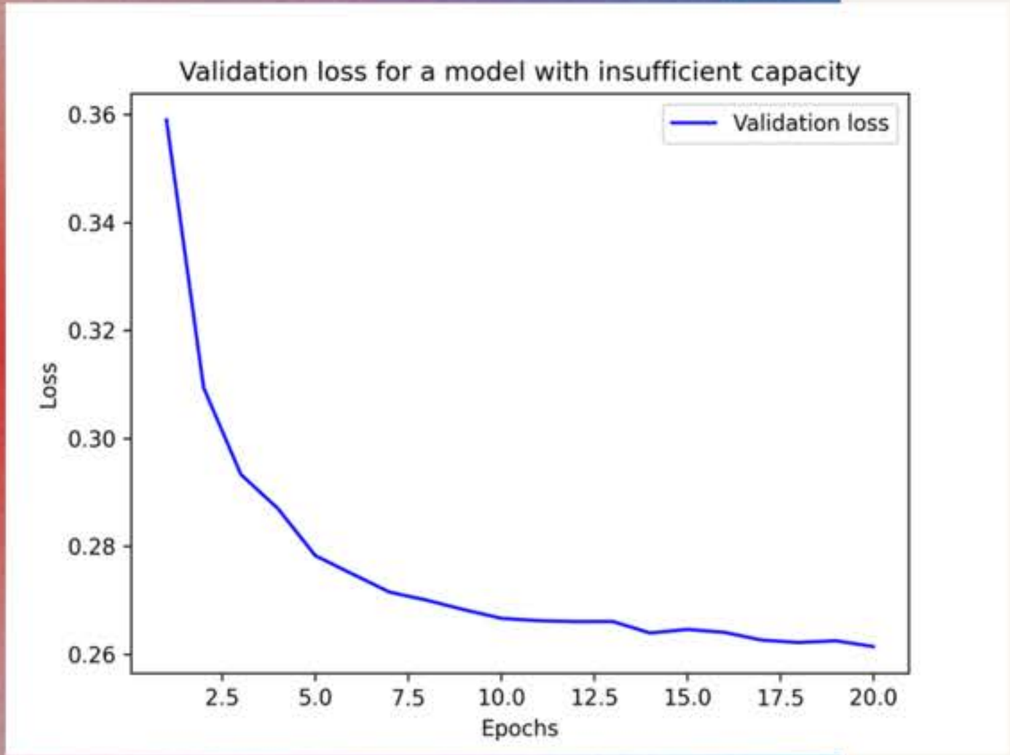
典型的過度配適現象

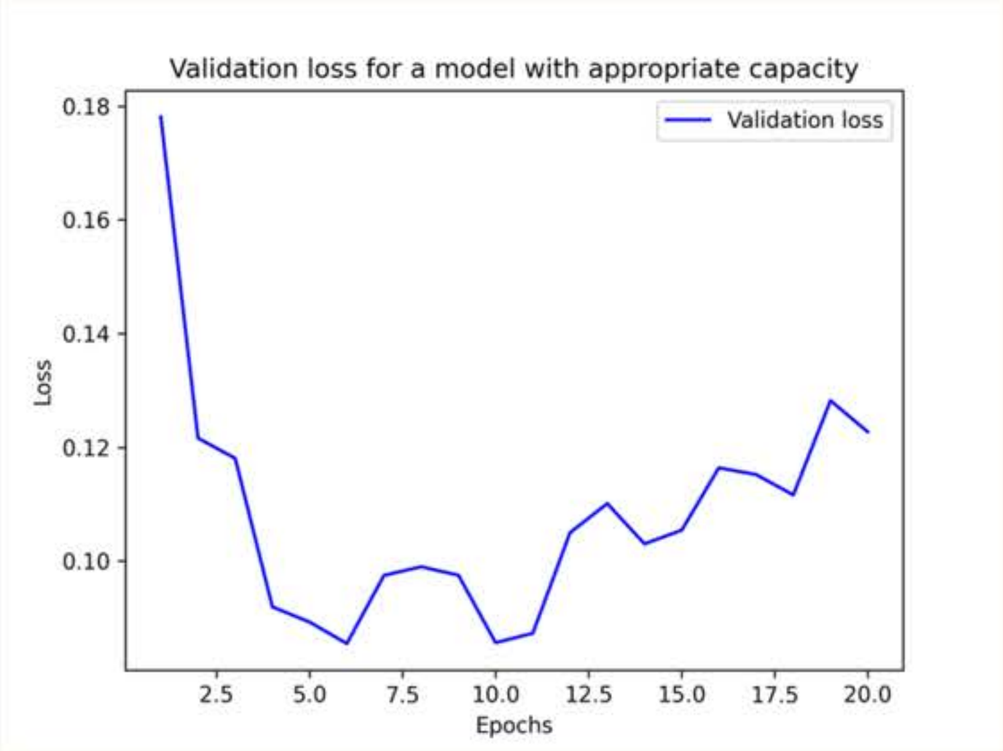


提升模型的擬合表現 (克服 Underfitting)

調整方向 3：模型容量（Capacity）

- 小模型問題： 若模型太簡單（層數少、神經元少），無法捕捉數據的複雜特徵（無法擬合流形）
- 解法： 增加層數或神經元數量（增加表示能力），直到 Training Loss 能夠順利下降





容量不足(卡在 0.26)

Loss 雖然有下降，但很快就卡住平緩了

容量適中（降至 0.09）

當我們增加層數或神經元後，Loss 突破了瓶頸，持續下降到更低的水準

提升普適化能力（克服 Overfitting）

關鍵轉折點

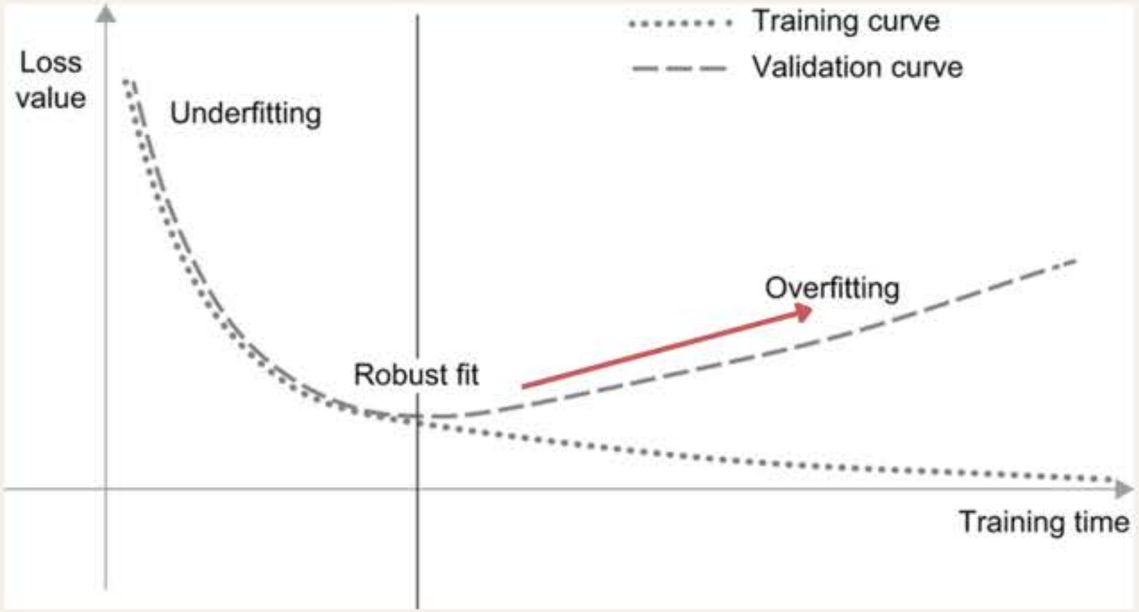
- 當模型容量足夠時，Training Loss 會持續下降
- 但當 Validation Loss開始不降反升時，代表模型開始死背訓練資料，即發生過度配適（Overfitting）

首要策略：數據

- 最佳解法： 獲取更多、品質更好的訓練資料
- 這是提升普適化能力最有效、也是最根本的方法

次要解法：常規化

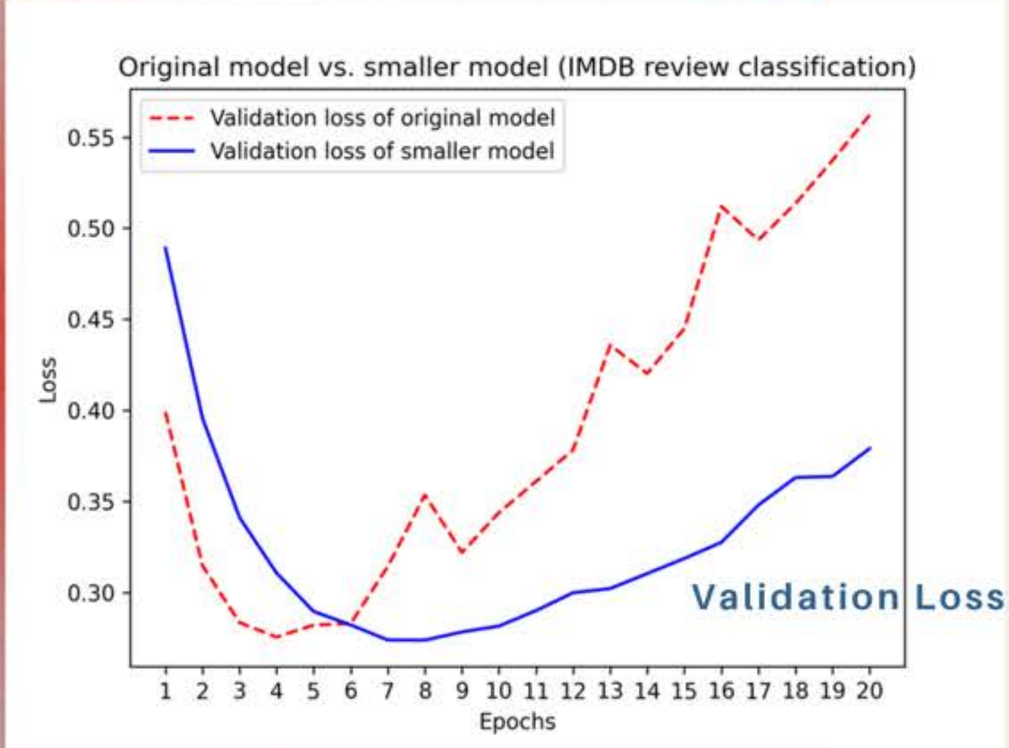
- 若數據有限，則需使用技術手段限制模型複雜度，強制模型學習更簡單的規律
- 技術手段有像是 L2 Regularization 或 Dropout...等等



常規化策略一：調整模型容量

實驗對比（IMDB 資料集）

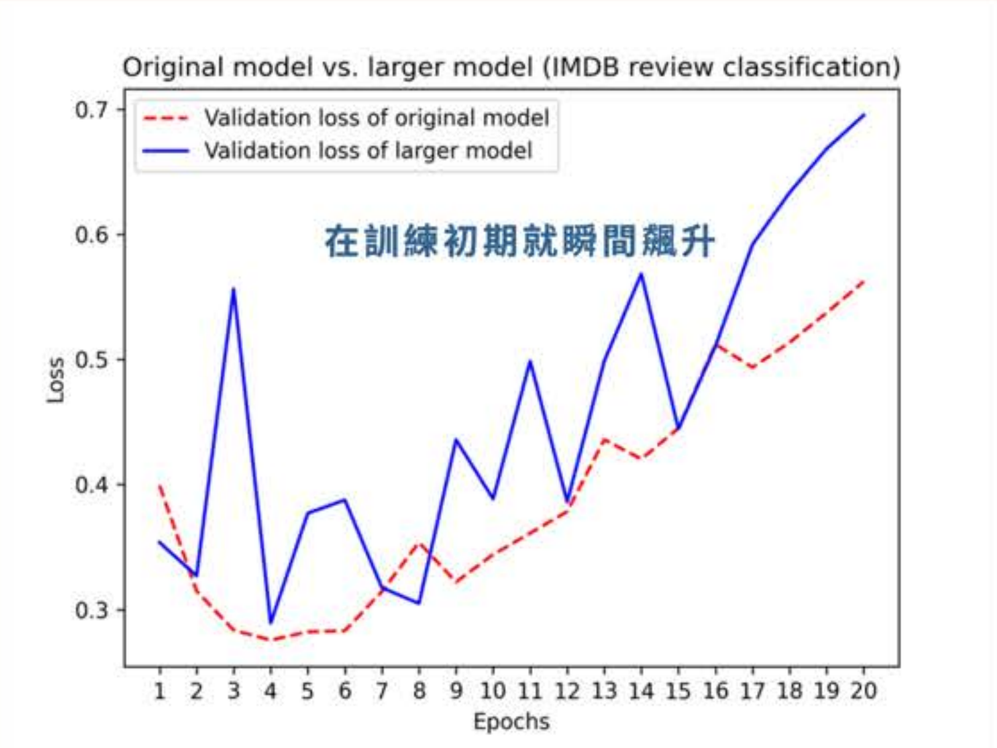
小容量模型(4 units)



Validation Loss 上升較緩慢

- 優點：較難發生過度配適
- 缺點：可能容量不足，導致擬合表現受限

大容量模型(512 units)



- 代表大模型記憶力超強，能秒殺訓練集
- 對於沒看過的資料完全無法應對

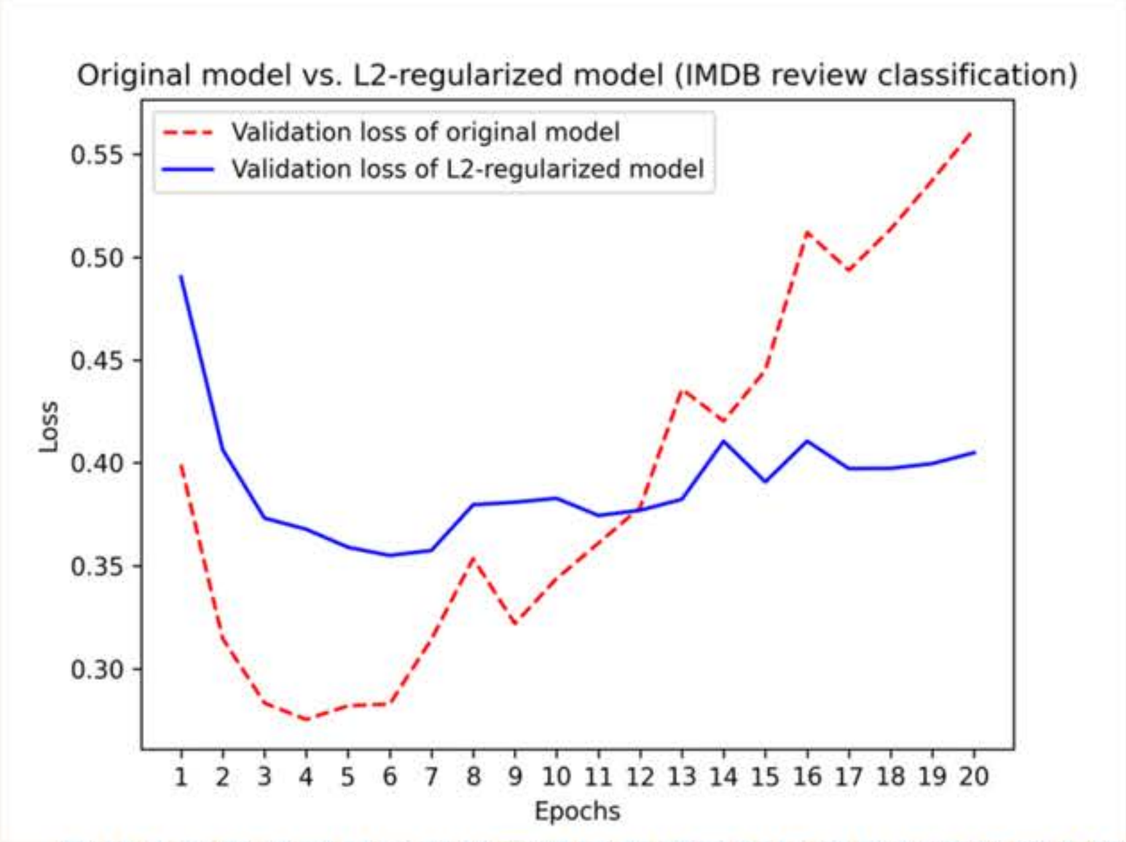
常規化策略二：權重常規化

原理：奧卡姆剃刀

- 給定兩種解釋，最簡單的那個通常是正確的
- 簡單的模型（參數量少或參數值小）較不易過度擬合

L2 Regularization

- 作法：在 Loss Function 中加入成本（Cost），強制讓權重保持微小



- 藍線上升幅度明顯比紅線平緩證明模型對抗過度配適的能力變強了

常規化策略三：Dropout 與 早期停止

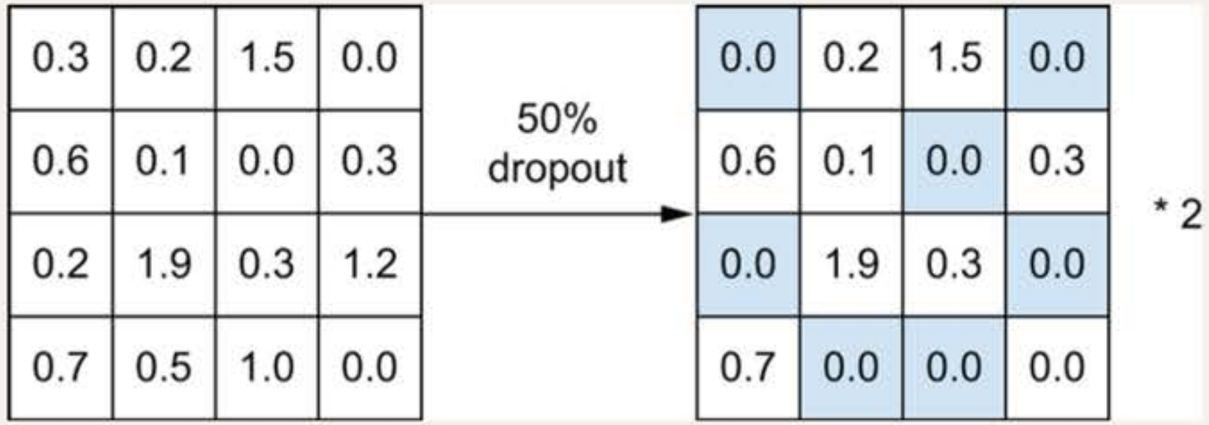
Dropout（丟棄法）

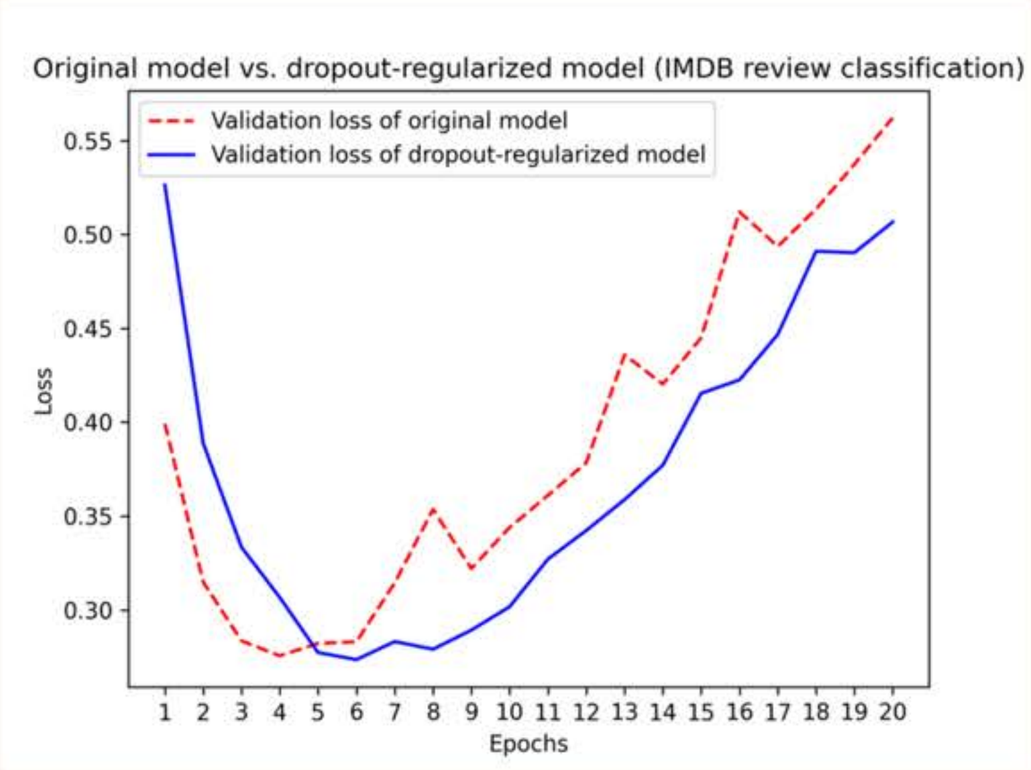
- 原理：訓練時隨機將部分神經元的輸出歸零（如左圖矩陣）
- 目的：打破神經元之間的共伴效應，強迫模型學習更分散、更魯棒的特徵
- 效果（右圖）：藍線的 Loss 明顯低於紅線，這是目前最有效的常規化技術之一

早期停止

- 不需要跑完所有 Epochs
- 一旦發現 Validation Loss 達到最低點（如右圖約 Epoch 4-9 處），即可停止訓練並保存模型

Dropout 原理





Dropout 效果

CH6
機器學習的通用工作流程

主講：邱偉豪

1 定義任務
(Defining the Task)

2 開發模型
(Developing a Model)

3 部署模型
(Deploying the Model)

相關檔案

- Deep Learning with Python

定義任務 (Defining the Task)

核心概念

- 在動手寫程式碼之前，必須先深入理解業務背景與數據限制

主要步驟

1. 構建問題框架 (Framing the problem)
   - 確認輸入數據與預測目標
   - 確認任務類型（分類、回歸等）
   - 了解現有解決方案與系統限制（如：加密需求、延遲限制）
2. 收集數據 (Collecting a dataset)
   - 數據標註：自行標註 vs. 外包
   - 注意非代表性數據 (Non-representative data)的陷阱（訓練數據需貼近生產環境）
3. 理解數據 (Understanding the data)
   - 分析數據特性，避免將偏差 (Bias) 帶入模型
4. 選擇成功指標 (Choosing a measure of success)
   - 準確率 (Accuracy)、精確率/召回率 (Precision/Recall) 或自定義指標。
   - 指標需直接對齊業務目標。

開發模型 (Developing a Model)

核心目標

- 建立一個能夠學習並具備泛化能力的模型

步驟 1：準備數據

- 向量化 (Vectorization)：將輸入轉為張量
- 值標準化 (Normalization)：調整數據範圍（如 0-1）
- 處理缺失值

步驟 2：選擇評估協議

- 留出驗證集 (Hold-out validation)
- K-折交叉驗證 (K-fold cross-validation)（適用於數據較少時）

步驟 3：超越基準

- 建立一個簡單的基準模型（如：隨機猜測、簡單統計平均值）
- 證明深度學習模型的表現優於這些簡單方法（具備統計功效）

開發模型
(Developing a Model)

步驟 4：開發過擬合的模型

- 原因：如果不先過擬合，就不知道模型的極限在哪（無法確認模型容量是否足夠）
- 方法：增加層數、增加神經元、訓練更多輪次
- 訊號：當訓練損失下降但驗證損失開始上升時，即達到此階段

步驟 5：正則化與調優

- 目標：將過擬合的模型轉化為最佳泛化模型
- 方法：
  - 減少網絡規模
  - 添加權重正則化（L1/L2）
  - 加入 Dropout
  - 調整超參數（Hyperparameters）

部署模型 (Deploying the Model)

核心概念

- 模型開發完成只是工作的一部分，如何讓模型在真實世界產生價值同樣重要

部署方式（Deploying）

- REST API：伺服器端運行（如 Flask/FastAPI）
- 設備端（On-device）：手機或嵌入式設備（低延遲、隱私考量）
- 瀏覽器端（Browser）：使用 TensorFlow.js 等

監控與維護 (Monitoring & Maintenance)

監控模型（Monitoring）

- A/B 測試：比較新舊模型的實際業務影響
- 定期審計：人工檢查模型在生產數據上的預測結果

維護模型（Model Maintenance）

- 應對概念漂移（Concept Drift）：隨著時間推移，數據分佈會發生變化（如：使用者行為改變、新詞彙出現）。
- 持續迭代：收集新的困難樣本進行標註、定期重新訓練模型以適應新數據